

Contents

Implementation Plan: Spatially-Varying Dynamic-Rupture Driver	1
Overview	1
Constraints	2
Phase 0: Friction-config TOML schema (slip-weakening half)	3
Phase 1: Stress-sidecar → FaultFaceFlux DOFData wiring	6
Phase 2: Heterogeneous fault-QP impedance (delivered, not deferred)	8
Phase 3: Velocity + stress bundle reuse	9
Phase 4: seas_spatial_dyn_driver — main driver	10
Phase 5: Existing-operator additive surface	13
Phase 6: Sbatch + verification	13
Testing Strategy	14
Risk Assessment	14
Out of Scope for This Plan	15

Implementation Plan: Spatially-Varying Dynamic-Rupture Driver

Scope generalisation note (2026-05-17): this driver is generic — any single-event dynamic-rupture simulation with spatially-varying material / pre-stress / friction can use it. The SAFS multi-fault San Andreas dataset under `safs/project_7.0_alternative/...` is the first concrete example exercised by the acceptance tests; the architecture and CLI carry no SAFS-specific assumptions. All driver, source, and test names use the `spatial` prefix; only dataset paths and SAFS-specific TOML/sbatch fills retain the `safs` infix to disambiguate which example dataset they target.

Date: 2026-05-17 Supersedes (in part): `heterogeneous_material_plan.md` Phase 3 (WaveOperator + GodunovFluxPool) — that plan is prerequisite, not duplicated. This plan layers driver wiring, restart, and ParaView output on top. Companion plan: `spatial_quasi_dynamic_plan.md` (quasi-dynamic sibling that shares the same mesh / velocity / stress / friction TOML schema).

Overview

Build a single executable `seas_spatial_dyn_driver` that runs an SCEC-style dynamic-rupture simulation on the SAFS multi-fault San Andreas geometry with all four data layers spatially varying: mesh, velocity, pre-stress, and slip-weakening (or rate-and-state) friction parameters. Same data sources, same TOML schema, same restart/ParaView wiring as `spatial_quasi_dynamic_plan.md`; the only differences are:

Aspect	QD sibling	This plan (dynamic)
Bulk operator	ElasticityDomainOperator<ParMesh> (DG, static equilibrium)	WaveOperator<ParMesh> (DG, ADER time-stepping)
Time-step	adaptive RK45 / PETSc-TS, Δt ranges yr → s	explicit ADER-O{N}, Δt fixed (CFL-limited, fractions of s)
Fault flux	RateStateFaultOperator reduced quasi-static	FaultFaceFlux::EvaluateADER_LSW or EvaluateADER (closed-form Riemann)
Friction default	rate-and-state (Dieterich-Ruina)	slip-weakening (LSW, TPV205-style) — both supported, user-selectable in TOML

Aspect	QD sibling	This plan (dynamic)
Material rep.	per-qpoint via <code>Coefficient::Eval</code> inside DG integrators	per-element via <code>GodunovFluxPool</code> (one cached Jacobian per unique (λ , μ , ρ) triple)
Output cadence	regime-aware (yr/s/ms)	fixed Δt -scaled with <code>--paraview-volume-dt</code>
Restart	V2 PETSc-TS	V1 TPV104-style (existing <code>io/tpv104_checkpoint.hpp</code>)
Wall-clock target	hours-days (yr-Myr sim time)	minutes-hours (seconds-of sim time)

The driver reuses every existing TPV104/205 building block (`WaveOperator`, `FaultFaceFlux::EvaluateADER_LSW`, `Tpv104SubStepIterator`, ADER predictor, `Tpv104Checkpoint`, the paraview-compaction `ParaViewOutput`). It adds only: 1. Wiring `MaterialField::Mode::Coefficient` into `WaveOperator` (delegated entirely to heterogeneous_material_plan.md Phase 3 — `GodunovFluxPool`). 2. SAFS-specific friction-parameter resolution (per-DOF LSW or rate-state vectors from a TOML sidecar that this plan shares with the QD sibling). 3. SAFS-specific initial stress (via existing `FaultGeometry::ComputeSAFSParams` + a `FaultFaceFlux::InitializeFromSpatialStress` setter added in Phase 5). 4. V1 checkpoint round-trip across a SAFS-scale mesh. 5. `--paraview-fault-hdf5` defaults that suit dynamic-rupture file sizes (snapshot every ms, ZFP fault tolerance $1e-12$).

Constraints

Interface constraints — what cannot change

- **`WaveOperator<ParMesh> (MaterialField)`** constructor from heterogeneous_material_plan.md Phase 3 is the only one used here. The legacy (λ , μ , ρ) constructor must remain bit-equivalent for non-SAFS TPV104/205 callers.
- **`FaultFaceFlux DOFData`** struct (in `dynamic/fault_face_flux.hpp:59–75`) is the per-fault-DOF data record. This driver populates LSW fields (`lsw_mu_s`, `lsw_mu_d`, `lsw_d_c`) for the slip-weakening path OR rate-state fields (`a`, `b`, `Dc`, `f_0`, `V_0`) for the RS path — never both; the misuse guard in `EvaluateADER_LSW` already enforces this at runtime.
- **`Tpv104SubStepIterator`** + ADER-O{N} predictor are the time integrators. Reused verbatim.
- **`Tpv104Checkpoint`** (V1 schema) is the restart mechanism. Reused verbatim with a new `driver_tag = "spatial_dyn"` field in the schema's metadata so a checkpoint cannot be restored into the wrong driver.
- **`ParaViewOutput<ParMesh>`** API as documented in the QD plan — same `InitFaultOutputBP5(...)`, `SetFaultOutputMode(FaultOutputMode::Hdf5)`, `SetHDFCompression(...)`. The same [output] TOML block applies.
- CLAUDE.md sign conventions apply unchanged: `dip = (0, 0, +1)` downward; $\sigma_n > 0$ compression; tangent frame `t1 = dip`, `t2 = strike` (FaultBasis Tandem convention). The TPV205 LSW solve already respects these conventions; SAFS reuses verbatim.
- **`FaultFaceFlux`** impedance computation — currently uses constant (λ , μ , ρ) from the operator (`InitializeImpedancesFromMaterial`). For SAFS with `Mode::Coefficient`, impedances must be per-fault-QP evaluated via `material.EvalAt(elem, T, ip, ...)`. This is the Phase 5 extension here and is the same TODO flagged in heterogeneous_material_plan.md Risk Assessment "Fault impedance computation".

Dependency constraints

- **heterogeneous_material_plan.md** Phases 1+3 must be merged first — driver consumes `MaterialField::MakeCoefficient(...)` and the `WaveOperator(MaterialField)` constructor. In the meantime the driver MAY operate with `MaterialField::MakeConstant(...)` from the friction-config TOML's `[material_constant_fallback]` block.
- **StressField3D** and **FaultGeometry::ComputeSAFSParams** as in the QD sibling.
- **extern/toml11** (already present).
- MFEM build flags: `MFEM_USE_HDF5=YES` (VTKHDF), `MFEM_USE_H5Z_ZFP=YES` (ZFP, strongly recommended for dynamic-rupture file sizes), `MFEM_USE_MPI=YES`. PETSc NOT required for V1 restart.
- CLAUDE.md “do not modify” zones: `friction/dieterich_ruina.hpp`, `solver/seas_operator.hpp`, `bp5/`, `bp1/`, `bp2/`. All driver code lives under `drivers/` and `spatial/code/`; the only existing operator touched is `FaultFaceFlux` (Phase 5) and `WaveOperator` (via the `heterogeneous_material_plan` Phase 3 work).

Convention constraints

- TOML for all configuration, same schema as `spatial_friction_config_schema.md` (Phase 0 of the QD plan). This driver requires `[meta].law == "slip_weakening"` OR `[meta].law == "rate_state"`; both are supported but slip-weakening is the production default for SAFS dynamic-rupture (per the TPV205-style precedent and the `barbot26c.pdf` reference in the friction directory).
- Output directory layout identical to the QD driver, except `volume.vtkhdf` now stores velocity ($\dot{u}$) and stress (σ) as the two registered fields, and the checkpoint file is named `tpv104_checkpoint.h5` (V1 schema — preserved name for tool compatibility).
- Naming: source files `safs_dyn_*.cpp/hpp` under `spatial/code/`; test files `tests/unit/test_safs_dyn_*.cpp`; sbatch under `jobs/safs/`.
- Greppable markers: every TODO this plan defers is tagged `// SPATIAL-DYN R-XXX TODO`.

Numerical constraints

- CFL: heterogeneous $CFL = cfl * h_e / cp_max_e$ per element, reduced with `MPI_Allreduce(MIN)` (already in `heterogeneous_material_plan.md` Phase 3 Detailed Req. 5).
- ADER order: default 2 (matches TPV104/205 production); user-overridable via `--ader-order N`.
- Mixed-flux mode: default none (upwind everywhere, TPV104 production default); options `adjacent` and `all-continuous` available via `--mixed-flux` flag (already in `WaveOperator`).
- T_{final} : typically seconds to tens of seconds for a single rupture event. Schema accepts "12s" or "0.01s".
- Fault-output cadence: default 1 ms (`--paraview-fault-dt 0.001`); volume cadence default 50 ms (`--paraview-volume-dt 0.05`). These defaults are pinned in the TOML schema; CLI overrides win.
- ZFP defaults per `miniapps/seas/CLAUDE.md` ParaView section: `paraview_volume_zfp_tol = 1e-3` (primary velocity), `paraview_bulk_zfp_tol = 1e-3` (secondary stress), `paraview_fault_zfp_tol = 1e-12` (slip-rate floor — orders smaller than bulk because slip rate spans $1e-9..1e1$ m/s during a single event).

Phase 0: Friction-config TOML schema (slip-weakening half)

Goal

Extend `spatial_friction_config_schema.md` (created by the QD plan's Phase 0) with the complete `[friction.slip_weakening]` block specification, and provide the example file the dynamic-rupture driver consumes.

Files to Create (or extend if shared with QD)

- safs/project_7.0_alternative/document/spatial_friction_config_schema.md — extend §2 with the full LSW block (the QD plan's Phase 0 left this as a forward-reference).
- safs/project_7.0_alternative/friction/EXAMPLE_spatial_friction_slip_weakening_safs.toml — already drafted in the QD plan's Phase 0 deliverable list; this plan populates the full body here.

Detailed Requirements

1. Slip-weakening block (TOML, completing what the QD plan stubbed):

```
[meta]
schema_version = 1
law            = "slip_weakening"
description    = "SAFS-ALT6 multi-fault slip-weakening, TPV205-style"

[material_constant_fallback]
lambda = 32.0e9
mu      = 32.0e9
rho     = 2670.0

[pore_pressure]
P_p_pa      = 0.0
P_p_grad_pa_per_m = 0.0
min_sigma_n_pa = 0.0

[time]
tfinal      = "12s"
t_initial   = 0.0
dt_initial  = "auto"      # "auto" => 0.5 * CFL_local_min
dt_max      = "0.1s"      # safety cap
cfl         = 0.5

[output]
output_dir      = "output_safs_dyn_500m"
paraview_volume = "hdf5"
paraview_fault  = "hdf5"
paraview_volume_dt = "0.05s"
paraview_bulk_dt  = "0.05s"
paraview_fault_dt  = "0.001s"
paraview_volume_zfp_tol = 1.0e-3
paraview_bulk_zfp_tol  = 1.0e-3
paraview_fault_zfp_tol  = 1.0e-12
max_snapshots      = 5000
checkpoint_every_steps = 10000

[friction.slip_weakening]
# Scalar defaults (applied everywhere unless a spatial rule overrides).
mu_s_default = 0.677
mu_d_default = 0.525
d_c_default  = 0.40      # m
cohesion_default = 0.0    # Pa
# Initial-stress override at the nucleation patch – overrides whatever
# the stress sidecar would otherwise produce in that region.
nuc_tau_override_pa = 81.6e6
```

```

nuc_sigma_n_override_pa = 120.0e6

# Spatial rules, applied in order, last-match wins per key.
[[friction.slip_weakening.spatial]]
kind          = "depth"
z_min_m       = -20000.0
z_max_m       = -15000.0
mu_s          = 1.0e6          # barrier (effectively infinite)

[[friction.slip_weakening.spatial]]
kind          = "box"          # nucleation patch
x_min_m       = ...
x_max_m       = ...
y_min_m       = ...
y_max_m       = ...
z_min_m       = ...
z_max_m       = ...
apply_nuc_override = true     # uses the [friction.slip_weakening] nuc*_override_pa

```

2. Validation rules (enforced in QD-Phase-1 parser, extended for LSW):

- $0 < \mu_d < \mu_s$, $d_c > 0$, cohesion ≥ 0 at every DOF after spatial application.
- $\mu_s == 1.0e6$ (the barrier sentinel from `dynamic/fault_face_flux.hpp:73` comment) marks a barrier DOF; the LSW closed-form solve treats this DOF as locked.
- At any DOF inside a `apply_nuc_override` box, τ_{pre} and σ_{n_eff} are replaced by the `nuc*_override_pa` scalars at the end of the resolver (overriding the stress sidecar).

3. Sharing the resolver: the QD plan's `SpatialFrictionResolver` already declares `ResolveSlipWeakening(...)`. Its body is specified here (the QD plan declared the interface only):

```

SlipWeakeningPerDOFParams SpatialFrictionResolver::ResolveSlipWeakening(
    const SpatialFrictionConfig& cfg,
    const Vector& dof_coords_3d,
    const Array<int>& dof_to_attr) const
{
    const int N = dof_coords_3d.Size() / 3;
    SlipWeakeningPerDOFParams p;
    p.mu_s.SetSize(N); p.mu_d.SetSize(N);
    p.d_c.SetSize(N); p.cohesion.SetSize(N);
    // 1. Seed defaults.
    p.mu_s = cfg.mu_s_default;
    p.mu_d = cfg.mu_d_default;
    p.d_c = cfg.d_c_default;
    p.cohesion = cfg.cohesion_default;
    // 2. Apply spatial rules in order.
    for (int i = 0; i < N; i++) {
        const real_t x = dof_coords_3d(3*i + 0);
        const real_t y = dof_coords_3d(3*i + 1);
        const real_t z = dof_coords_3d(3*i + 2);
        for (const auto& r : cfg.spatial) {
            if (!r.matches(x, y, z, dof_to_attr[i])) { continue; }
            if (!std::isnan(r.mu_s)) { p.mu_s(i) = r.mu_s; }
            if (!std::isnan(r.mu_d)) { p.mu_d(i) = r.mu_d; }
            if (!std::isnan(r.d_c)) { p.d_c(i) = r.d_c; }
            if (!std::isnan(r.cohesion)) { p.cohesion(i) = r.cohesion; }
        }
    }
}

```

```

    // 3. Validate.
    MFEM_VERIFY(p.mu_s(i) > p.mu_d(i),
        "LSW resolver: mu_s <= mu_d at DOF " << i);
    MFEM_VERIFY(p.d_c(i) > 0.0,
        "LSW resolver: d_c <= 0 at DOF " << i);
    MFEM_VERIFY(p.cohesion(i) >= 0.0,
        "LSW resolver: cohesion < 0 at DOF " << i);
}
return p;
}

```

Acceptance Criteria

- ☐ EXAMPLE_spatial_friction_slip_weakening_safs.toml parses through the QD-plan-Phase-1 parser.
- ☐ test_spatial_friction_resolver (8 tests, QD plan) gains 4 LSW-specific tests: default scalar, depth barrier rule, box nucleation override, validation aborts.
- ☐ Schema doc cross-references the dynamic-rupture driver and the LSW solver.

Dependencies

- Depends on: QD plan Phase 0 (shared schema doc + spatial-rule data structure) and QD plan Phase 1 (SpatialFrictionResolver already includes the ResolveSlipWeakening declaration).
- Required by: Phase 4 of this plan.

Phase 1: Stress-sidecar → FaultFaceFlux DOFData wiring

Goal

Take `geom.HasSAFSParams() == true` (populated by `ApplySpatialStressSidecar` from the QD plan's Phase 3) and push the per-DOF `tau_pre / sigma_n` into `FaultFaceFlux::DOFData[i]` so the existing `EvaluateADER_LSW` closed-form solve consumes SAFS-realistic stress without any further change to the solver.

Files to Create

- `spatial/code/spatial_dyn_fault_setup.hpp / .cpp` — translates `FaultGeometry` per-DOF arrays into `FaultFaceFlux::DOFData[]`.

Files to Modify

- `dynamic/fault_face_flux.hpp`: add only a public setter (additive; CLAUDE.md “rest of dynamic/editable” applies):

```

/// SAFS dynamic-rupture init: populate per-DOF (tau1_0, tau2_0, sigma_n_eff,
/// lsw_mu_s, lsw_mu_d, lsw_d_c) from caller-supplied vectors. Sizes must
/// equal num_fault_dofs; tau_pre is interleaved [dip, strike] of length
/// 2 * num_fault_dofs. Cohesion is optional (size 0 → zero everywhere).
/// Aborts if called twice.
void InitializeFromSpatialStress(const Vector& tau_pre_per_dof,
                                const Vector& sigma_n_eff_per_dof,
                                const Vector& lsw_mu_s_per_dof,
                                const Vector& lsw_mu_d_per_dof,

```

```

        const Vector& lsw_d_c_per_dof,
        const Vector& lsw_cohesion_per_dof);

/// Rate-state alternative; same shape but six R-S fields per DOF.
void InitializeFromSpatialStressRateState(const Vector& tau_pre_per_dof,
        const Vector& sigma_n_eff_per_dof,
        const Vector& a_per_dof,
        const Vector& b_per_dof,
        const Vector& Dc_per_dof,
        const Vector& f_0_per_dof,
        const Vector& V_0_per_dof,
        const Vector& V_init_per_dof);

```

Detailed Requirements

1. Implementation skeleton (slip-weakening case):

```

void FaultFaceFlux::InitializeFromSpatialStress(...) {
    MFEM_VERIFY(!spatial_initialized_, "FaultFaceFlux: InitializeFromSpatialStress called twice");
    MFEM_VERIFY(tau_pre_per_dof.Size() == 2 * num_fault_dofs_, "...");
    MFEM_VERIFY(sigma_n_eff_per_dof.Size() == num_fault_dofs_, "...");
    MFEM_VERIFY(lsw_mu_s_per_dof.Size() == num_fault_dofs_, "...");
    // ... validate all sizes ...

    for (int i = 0; i < num_fault_dofs_; i++) {
        dof_data_[i].tau1_0 = tau_pre_per_dof(2*i + 0); // dip
        dof_data_[i].tau2_0 = tau_pre_per_dof(2*i + 1); // strike
        dof_data_[i].sigma_n = sigma_n_eff_per_dof(i);
        dof_data_[i].lsw_mu_s = lsw_mu_s_per_dof(i);
        dof_data_[i].lsw_mu_d = lsw_mu_d_per_dof(i);
        dof_data_[i].lsw_d_c = lsw_d_c_per_dof(i);
        dof_data_[i].cohesion = lsw_cohesion_per_dof.Size() > 0
            ? lsw_cohesion_per_dof(i) : 0.0;
        // RS fields stay zero (the misuse guard in EvaluateADER_LSW
        // only fires if both LSW and RS are populated).
    }
    spatial_initialized_ = true;
}

```

2. spatial_dyn_fault_setup.hpp exposes the glue:

```

namespace mfem::seas::spatial {
    /// One-shot: pre-stress + friction → FaultFaceFlux dof_data_.
    /// Asserts geom.HasSAFSPParams() == true; reads from
    /// geom.{sigma_n_per_dof(), and a parallel tau_pre vector built by
    /// ComputeSAFSPParams. Resolved per-DOF LSW params are supplied by
    /// the caller (driver computes them via SpatialFrictionResolver).
    void InitFaultFaceFluxFromSpatial(FaultGeometry<ParMesh>& geom,
        const SlipWeakeningPerDOFParams& lsw,
        FaultFaceFlux& ffl);
    void InitFaultFaceFluxFromSpatialRateState(FaultGeometry<ParMesh>& geom,
        const RateStatePerDOFParams& rs,
        FaultFaceFlux& ffl);
}

```

3. Edge cases:

- Empty `geom.sigma_n_per_dof()` (SAFS sidecar not applied) → MFEM_ABORT with “InitFaultFaceFluxFromSpatial: geom.HasSAFSParams() returned false; call ApplySpatialStressSidecar first.”
- LSW barrier DOF (`mu_s == 1.0e6`) → just pass through; the closed-form solver already handles it.
- Nucleation override (per Phase 0’s `apply_nuc_override`) — the driver applies the override to `sigma_n_eff` and `tau_pre` BEFORE calling `InitFaultFaceFluxFromSpatial`; the setter is unaware of the nuc patch.

Acceptance Criteria

- ☐ New test `test_spatial_dyn_fault_setup_lsw`: 5 tests covering size validation, default LSW write-through, barrier DOF, double-init abort, missing stress-sidecar abort.
- ☐ New test `test_spatial_dyn_fault_setup_rs`: 4 tests, same shape with RS fields.
- ☐ No existing TPV104/205 test regresses (the new setters are additive; existing init paths untouched).

Dependencies

- Depends on: QD plan Phase 3 (`ApplySpatialStressSidecar`); QD plan Phase 1 (`SlipWeakeningPerD0FParams`, `RateStatePerD0FParams`); existing `FaultFaceFlux`.
- Required by: Phase 4.

Phase 2: Heterogeneous fault-QP impedance (delivered, not deferred)

Goal

Replace the existing `FaultFaceFlux::InitializeImpedancesFromMaterial(scalar_lambda, scalar_mu, scalar_rho)` with a per-fault-QP variant that calls `material.EvalAt(elem, T, ip, ...)` so heterogeneous SAFS impedances are correct. This is the TODO that heterogeneous_material_plan.md Risk Assessment flagged as “Low-confidence; investigation needed” — the SAFS dynamic-rupture path forces us to resolve it now.

Files to Modify

- `dynamic/fault_face_flux.hpp`: add a NEW setter (existing scalar `InitializeImpedancesFromMaterial` is left in place for TPV104/205 backward compatibility — both paths coexist):

```
/// Per-fault-QP impedance initialisation for heterogeneous SAFS runs.
/// For each fault QP i, evaluates lambda/mu/rho via material at the
/// QP's (elem, T, ip) and stores eta_p(i), eta_s(i) per DOF.
/// The existing scalar setter remains for non-SAFS callers.
void InitializeImpedancesPerQP(const MaterialField& material,
                               ParMesh& mesh,
                               const Vector& dof_coords_3d,
                               const Array<int>& dof_to_elem);
```

Body:

```
void FaultFaceFlux::InitializeImpedancesPerQP(...) {
    const int N = num_fault_dofs_;
    eta_p_per_dof_.SetSize(N);
    eta_s_per_dof_.SetSize(N);
    for (int i = 0; i < N; i++) {
        const int e = dof_to_elem[i];
```



```

    ElementTransformation* T = mesh.GetElementTransformation(e);
    IntegrationPoint ip;
    mfem::Vector phys(3);
    phys(0) = dof_coords_3d(3*i + 0);
    phys(1) = dof_coords_3d(3*i + 1);
    phys(2) = dof_coords_3d(3*i + 2);
    T->TransformBack(phys, ip);           // physical → reference
    real_t lam, mu, rho;
    material.EvalAt(e, *T, ip, lam, mu, rho);
    eta_p_per_dof_(i) = std::sqrt((lam + 2.0 * mu) * rho);
    eta_s_per_dof_(i) = std::sqrt(mu * rho);
}
impedances_per_qp_active_ = true;
}

```

- All call sites in `fault_face_flux.cpp` that read `eta_p_ / eta_s_` (scalars) become:

```

const real_t eta_p_i = impedances_per_qp_active_ ? eta_p_per_dof_(i) : eta_p_;
const real_t eta_s_i = impedances_per_qp_active_ ? eta_s_per_dof_(i) : eta_s_;

```

Same routing pattern as the SAFS-mode work in `rate_state_fault.hpp`.

Detailed Requirements

1. Bit-exact non-SAFS path: when `impedances_per_qp_active_ == false` (every TPV104/205 run), code reads exactly the same `eta_p_/eta_s_` scalars as today. Pinned by existing TPV104/205 regression.
2. Constant-as-heterogeneous parity: if the user passes a `MaterialField::MakeConstant( $\lambda$ ,  $\mu$ ,  $\rho$ )` through the new per-QP setter, every `eta_*_per_dof_(i)` equals the corresponding scalar. Pinned by a new test.
3. `MaterialField` lifetime: `InitializeImpedancesPerQP` only DEREFERENCES `material.EvalAt`; it does not store a Coefficient pointer. Once initialised, the eta vectors are independent of `MaterialField` lifetime.

Acceptance Criteria

- ☐ All existing TPV104/205 tests pass bit-identically (the new setter is additive; not called by those drivers).
- ☐ New test `test_fault_face_flux_per_qp_impedance`: 3 tests — constant-parity, layered sidecar produces per-DOF variation matching $\sqrt{\mu \cdot \rho}$ at the DOF coord, lifetime independence.
- ☐ Heterogeneous-material plan's Risk Assessment.Low-confidence.Fault impedance computation is now CLOSED; cross-reference in the next REVIEW.md.

Dependencies

- Depends on: `heterogeneous_material_plan` Phase 1.
- Required by: Phase 4.

Phase 3: Velocity + stress bundle reuse

Goal

No new code. Re-document that this plan reuses Phases 2 (`LoadSpatialVelocityBundle`) and 3 (`ApplySpatialStressSidecar`) from the QD plan.

Detailed Requirements

- The two bundles are identical for both drivers; the only difference is what comes after:
 - QD: `stress → RateStateFaultOperator::SetSAFSMode(true, ...)`.
 - Dynamic: `stress → FaultFaceFlux::InitializeFromSpatialStress[RateState](...)` (Phase 1 here).

Acceptance Criteria

- ☐ Smoke: `LoadSpatialVelocityBundle(...)` on the SAFS 1000m mesh + cvmh sidecar produces a `MaterialField::Mode::Coefficient` with all three pointers non-null. Same test as QD plan's Phase 2 acceptance.

Dependencies

- Depends on: QD plan Phases 2-3.
 - Required by: Phase 4.
-

Phase 4: **seas_spatial_dyn_driver** — main driver

Goal

Single executable that ties Phases 0-3 + heterogeneous WaveOperator + LSW (or RS) closed-form solver + ParaView VTKHDF + V1 restart into one runnable program.

Files to Create

- `drivers/spatial_dyn_driver.cpp` — the main. Layout mirrors `tpv104_driver.cpp` extensively (it shares the entire ADER + LSW + paraview + V1 restart machinery). ~1100 LOC expected.
- `spatial/code/spatial_dyn_driver_init.hpp / .cpp` — extracted init helpers (parse TOML, build bundles, construct ops, validate stress orientation).

Files to Modify

- `miniapps/seas/Makefile`:
 - Add `SPATIAL_DYN_DRIVER_SRC = drivers/spatial_dyn_driver.cpp`.
 - Add object + link rule (ends with `$(SEAS_POST_LINK_DEDUP_RPATH)` per Makefile R-005).
 - New umbrella test-`spatial-dyn-driver`: `seas_spatial_dyn_driver` (builds + `--dry-run`).
- `miniapps/seas/.gitignore`: add `seas_spatial_dyn_driver` to the binary list.

Interfaces

The driver's CLI:

```
seas_spatial_dyn_driver --config PATH.toml [OPTIONS]
```

Required:

```
--config PATH.toml          SAFS dynamic-rupture configuration
```

Mesh/data overrides (otherwise read from TOML):

```
--mesh PATH.msh
--velocity-model {cvmh|cvm_s4.26.m01|multiscale_statewise|<path>}
```

```
--stress-sidecar PATH.h5
--friction-config PATH.toml
```

Time / numerics:

```
--tfinal SECONDS|"12s"
--cfl FLOAT
--ader-order N           Default 2 (TPV104/205 production)
--mixed-flux {none|adjacent|all-continuous} Default none
```

Output (override [output]):

```
--paraview-volume {hdf5|vtu|off}
--paraview-fault {hdf5|vtu|off}
--paraview-volume-dt SECONDS Default 0.05s
--paraview-fault-dt SECONDS Default 0.001s
--paraview-volume-zfp-tol TOL
--paraview-fault-zfp-tol TOL
--paraview-max-snapshots N
```

Restart:

```
--restart PREFIX           Resume from <PREFIX>/tpv104_checkpoint.h5 (V1)
--checkpoint-every N
```

Diagnostics:

```
--dry-run           Parse + construct + print derived numbers; do not time-step
--verify-dispatch
--print-derived      After init, print L_nuc estimate (LSW only), CFL Δt min/max,
                    per-region histograms of mu_s/mu_d/d_c.
```

Detailed Requirements

main() order (each step greppable):

1. MPI init — same as `tpv104_driver.cpp`.
2. CLI parse + TOML load (`LoadSpatialFrictionConfig` from QD-Phase-1). Validate `meta.law ∈ {slip_weakening, rate_state}`; abort if the QD-only `rate_state` is mismatched against a CLI-supplied `--law slip_weakening`.
3. Output-dir / restart-dir safety checks (`weakly_canonical`, same as `tpv104`).
4. Load mesh → `ParMesh pmesh`. Apply the `BoundaryConfig` for the SAFS .geo (`fault_attr=101`, `top=102`, `bottom=103`, `sides=104`).
5. Build material (`LoadSpatialVelocityBundle` from QD-Phase-2 or fallback constant).
6. Construct **WaveOperator<ParMesh>** with the `MaterialField` constructor (`heterogeneous_material_plan` Phase 3). `[mesh].order` from TOML (default 1). CFL hard-coded into the operator's `ComputeMaxDt` walks the heterogeneous `c_p`; the driver fetches `cfl_dt = wave.ComputeMaxDt(toml.time.cfl)`.
7. Construct **FaultGeometry<ParMesh>** (BP5 ctor with the SAFS-aware `ElasticityDomainOperator`). Note: dynamic-rupture still uses `ElasticityDomainOperator` for fault-DOF bookkeeping (matching TPV104's pattern), even though the bulk physics goes through `WaveOperator`. Construct a throw-away `ElasticityDomainOperator` with the same `MaterialField` purely to provide the fault-DOF coordinate / basis arrays; this matches TPV104's existing two-operator pattern.
8. Apply stress sidecar (`ApplySpatialStressSidecar` from QD-Phase-3).
9. Resolve per-DOF friction params (QD-Phase-1 `SpatialFrictionResolver`). If `meta.law == slip_weakening`, call `ResolveSlipWeakening`; else `ResolveRateState`.
10. Initialise `FaultFaceFlux` (Phase 1 here): `InitFaultFaceFluxFromSpatial(geom, lsw_or_rs, wave.GetFaultFaceFlux())`.
11. Initialise per-QP impedances (Phase 2 here): `wave.GetFaultFaceFlux().InitializeImpedancesPerQP(materials)`.

- pmesh, geom.fault_dof_coords_3d(), elem_owners).
12. If **--dry-run** — print L_{nuc} estimate, Δt min/max, mesh stats, output paths; exit 0.
 13. Construct **ParaViewOutput<ParMesh>** — register velocity (primary pv_dc_ volume collection) and stress (secondary pv_bulk_out collection). Init fault output BP5. Apply paraview-compaction VTKHDF defaults + ZFP tolerances.
 14. Restart path — if **--restart PREFIX**, restore via `Tpv104Checkpoint::Restore("<PREFIX>/tpv104_checkpoint.state, t0, dt0, schedule_state)`. Otherwise $t_0=0$, $dt_0=cfl_dt$. Suppress initial $t=0$ station write on restart (paraview-compaction's existing fix, commit 6bd963f).
 15. Time loop — ADER-O{N} via `Tpv104SubStepIterator::Step(state, dt0, ...)`. Each step:
 - `pv.UpdateFaultFieldsBP5(...)` and `pv.WriteIfDue(t)` per fault cadence.
 - Volume + bulk PV writes per their cadences.
 - Station probe writes (existing TPV station list reused; SAFS-specific stations added via a [stations] TOML block in a follow-up).
 - Every `checkpoint_every_steps`, `Tpv104Checkpoint::Save(...)`.
 16. Finalisation — flush, final checkpoint, MPI finalise.

Edge Cases to Handle

- Friction TOML specifies **rate_state** law — driver runs with the RS LSW-less path (`EvaluateADER` rather than `EvaluateADER_LSW`). Both are supported.
- Stress sidecar bbox does not contain mesh — abort with the same precise message as the QD driver (single source of truth: `FieldProjector::AbortContainmentFailure`).
- **--restart PREFIX** schema mismatch — checkpoint contains `driver_tag` metadata; if `driver_tag != "spatial_dyn"`, abort. This requires adding a 32-byte `driver_tag` string to the V1 checkpoint schema header — additive, length-prefixed, defaults to "tpv104" for back-compat with existing TPV checkpoints.
- Bi-material face neighbour MPI exchange (GodunovFluxPool Phase 3 Detailed Req. 7a from `heterogeneous_material_plan`) — required at SAFS partition boundaries; passes the new `heterogeneous_material_plan` test `T-WAVEOP-BIMATERIAL-SHARED-FACE-MPI` which this driver implicitly exercises.

Acceptance Criteria

- ☐ make `seas_spatial_dyn_driver` builds; make `test-spatial-dyn-driver` runs **--dry-run** successfully on the 1000m_lcfar3000 mesh + cvmh sidecar + stress sidecar + the LSW EXAMPLE TOML, exits 0.
- ☐ `mpirun -np 8 seas_spatial_dyn_driver --config ... --tfinal 0.5s --dry-run` builds the same configuration in parallel and exits 0.
- ☐ V1 checkpoint round-trip: write at $t=0.5s$, restart, run to $t=1.0s$; compare `volume.vtkhdf` final snapshot against a continuous run's $t=1.0s$ snapshot; assert $L_{\infty} < ||\text{snapshot}||_{\text{inf}} * 1e-10$.
- ☐ Single-event smoke run (**--tfinal 1.0s**) writes a non-empty `volume.vtkhdf`, `bulk.vtkhdf`, and `fault.vtkhdf`; total output ≤ 5 GB.
- ☐ `verify_spatial_dyn_smoke_safs.py` (Phase 6) confirms slip-rate $\geq 1e-3$ m/s in the rupture core during the event.

Dependencies

- Depends on: Phases 0-2; QD plan Phases 1-3, 5; `heterogeneous_material_plan.md` Phases 1+3; existing TPV104 paraview + V1 checkpoint.
- Required by: nothing (leaf executable).

Phase 5: Existing-operator additive surface

Goal

Same intent as the QD plan's Phase 5: small additive accessor / setter additions. All living within "rest of dynamic/ is editable" per CLAUDE.md.

Files to Modify

- `dynamic/wave_operator.hpp`: expose

```
FaultFaceFlux& GetFaultFaceFlux();           // already present? if not, add  
const Array<int>& GetFaultD0FElemOwners() const; // new – bulk elem index per fault D0F
```
- `dynamic/fault_face_flux.hpp`: the additive setters from Phases 1 and 2 above.

Detailed Requirements

- All changes are header-only or in `.cpp` paired with header declarations; no constructor signature changes.
- Constant-mode regression covered by existing TPV104/205 tests.

Acceptance Criteria

- ☐ Existing `make test-tpv104` (55 tests, currently passing in REVIEW.md sweep) remains 55/55.
- ☐ Existing `make test-tpv102-local` continues to pass (currently fails on the SAFS branch for unrelated reasons; this plan does not regress it further).

Dependencies

- Depends on: nothing.
 - Required by: Phases 1, 2, 4.
-

Phase 6: Sbatch + verification

Goal

Production-ready sbatch for SAFS dynamic-rupture runs on Frontera + a small Python verification.

Files to Create

- `jobs/safs/spatial_dyn_smoke_8N_400r_dev_2hr_safs.sbatch` — Frontera dev queue, 8 nodes, 400 ranks, 2 hr wall, `--tfinal 0.5s` (a short rupture for end-to-end smoke).
- `jobs/safs/spatial_dyn_production_normal_48hr_safs.sbatch` — Frontera normal queue, 32 nodes, 48 hr, `--tfinal 30s` with V1 restart every 10000 steps.
- `spatial/code/scripts/verify_spatial_dyn_smoke_safs.py` — opens `<output_dir>/fault.vtkhdf`, asserts max slip-rate in rupture core $\geq 1e-3$ m/s, writes a 1-page text summary.

Detailed Requirements

1. sbatch ends in `$(SEAS_POST_LINK_DEDUP_RPATH)` via the new umbrella target (no `LC_RPATH` duplicate on Frontera).
2. **`verify_spatial_dyn_smoke_safs.py`**: `stdlib` + `h5py`; reads `fault.vtkhdf`, finds slip-rate field, computes max over all cells $\times$ all snapshots. Threshold and rupture-core region defined inline.

3. Document resource budget in each sbatch header (cores, peak RSS, wall, output volume).

Acceptance Criteria

- ☐ Both sbatches lint-pass `sbatch --test-only`.
- ☐ `verify_spatial_dyn_smoke_safs.py` runs cleanly on a committed reference under `tests/fixtures/safts_dyn/`

Dependencies

- Depends on: Phase 4.
- Required by: production runs.

Testing Strategy

Layer	Test	When
TOML LSW schema	extends test_spatial_friction_config (QD plan) +4 LSW tests	Phase 0
LSW resolver	extends test_spatial_friction_resolver (QD plan) +4 LSW tests	Phase 0
Fault setup	test_spatial_dyn_fault_setup (5), test_spatial_dyn_fault_setup_rs (4)	Phase 1
Per-QP impedance	test_fault_face_flux_per_qp_impedance (3)	Phase 2
Driver init	make test-spatial-dyn-driver runs --dry-run	Phase 4
Restart	V1 checkpoint round-trip inside test-spatial-dyn-driver umbrella	Phase 4
Smoke	verify_spatial_dyn_smoke_safts.py on a 1s Frontera run	Phase 6
Cross-plan	T-WAVEOP-BIMATERIAL- SHARED-FACE-MPI (heterogeneous_material_plan Phase 3)	Implicitly via Phase 4 driver run

Risk Assessment

High-confidence (low risk)

- Phases 1-3 are mostly mechanical: TOML wrangling and per-DOF vector population.
- Phase 5 additive accessors mirror the QD plan's Phase 5 pattern, proven safe by existing regression tests.

Medium-risk

- Phase 2 per-QP impedance is the single deepest semantic change in this plan. The new setter is additive and inactive unless called, but its presence inside `EvaluateADER_LSW` (the closed-form solver hot loop) needs careful audit to ensure the `impedances_per_qp_active_` branch is correctly routed in every site that reads `eta_p_ / eta_s_`. Mitigation: every call-site rewrite is a single line; the bit-exact TPV104/205 regression catches any miss.
- **GodunovFluxPool** per-element memory on a 1000m SAFS mesh. The mesh contains $\approx 4\text{M}$ tets; `heterogeneous_material_plan` Phase 3 Acceptance Criteria pins the budget at $\leq 2\text{ GB}$ heap. If the SAFS sidecar produces more unique (λ, μ, ρ) triples than expected (basin sediments smoothly variable), this can over-shoot. Mitigation: `dedup_sig_figs` is CLI-tunable; the plan documents the fallback to `dedup_sig_figs = 4`.

Low-confidence (need investigation before implementing)

- **Tpv104SubStepIterator** interaction with heterogeneous CFL. The sub-step iterator currently assumes a single global Δt . With heterogeneous CFL, Δt may need to be the per-step `MaxDt` re-evaluated rather than the construction-time value. Investigation: read `dynamic/tpv104_substep_iterator.cpp::Step` to confirm Δt is reused vs. re-fetched. If reused, add a `RefreshDt` call at the top of each macro step.
- V1 checkpoint **driver_tag** schema extension — adding a string field is additive, but every existing TPV104 checkpoint file on disk needs to round-trip without abort. Mitigation: default `driver_tag = "tpv104"` if the field is absent in the file; aborts only when both file and runtime declare different non-empty tags.
- Rupture nucleation tuning for SAFS-scale fault. TPV205 nucleation patches are calibrated for $\sim 30\text{ km}$ faults; SAFS multi-segment is $\sim 500\text{ km}$. The `[friction.slip_weakening.spatial.apply_nuc_override]` mechanism lets the user place the nucleation patch precisely, but no plan-internal physical correctness is asserted — the user is responsible for choosing values that nucleate. Mitigation: `--print-derived` reports $L_{\text{nuc}} = \mu * d_c / (\mu_s - \mu_d) / (\sigma_{\text{n_eff}})$ per region so the user can sanity-check.

Known tricky areas in existing code

- TPV104 mixed-flux dispatch (`dynamic/wave_operator.inl` `mixed_flux` switch) — heterogeneous mode means each side of an interior face evaluates `flux_pool_->At(elem)`. This is already covered by `heterogeneous_material_plan` Phase 3 Detailed Req. 6 (average flux).
- Restart from TPV104 V1 checkpoint into `spatial_dyn_driver` — guarded by the `driver_tag` extension; do not allow silent cross-driver restore.

Out of Scope for This Plan

1. Exact bi-material Riemann solver at heterogeneous interior faces. We continue to use the average-flux approximation from `heterogeneous_material_plan` Phase 3. Documented as a future-work item shared with that plan.
2. Pore-pressure spatial sidecar (constant + depth-gradient only, same as QD plan).
3. Time-dependent / damage material — material is static throughout the event.
4. GPU offload — same constraint as the QD plan (`MaterialField::EvalAt` is CPU-only).
5. Off-fault station network — initial release reuses TPV104 stations; SAFS-specific stations TBD.
6. Multi-event / cycle simulation — single-event only. Multi-event cycles belong on the QD path (`spatial_quasi_dynamic_plan.md`).
7. Slip-weakening on QD path — by design (the QD radiation-damping approximation pairs naturally with rate-and-state, not LSW); the QD driver aborts on LSW TOML and points at this plan.